

Radiology Assistant (MediScan AI) - Project Documentation

Radiology Assistant (MediScan AI)

- 1. Project Title
- 2. Project Overview
- 3. Problem Statement
- 4. Objectives
- 5. Dataset Description
- 6. Data Preprocessing
- 7. Exploratory Data Analysis
- 8. Data Augmentation
- 9. Model Architecture(s)
- 10. Training Process
- 11. Hyperparameters
- 12. Loss Function and Optimizer
- 13. Evaluation Metrics
- 14. Results
- 15. Visualizations
- 16. Inference / Prediction Process
- 17. Libraries and Dependencies
- 18. Project Folder Structure
- 19. Challenges Encountered
- 20. Future Improvements
- 21. Conclusion
- Appendix: Data Sources Referenced in the Notebooks

Radiology Assistant (MediScan AI)

A Multi-Modal Deep Learning System for Brain Tumor, Pneumonia, and COVID-19 Screening

1. Project Title

Radiology Assistant (MediScan AI) — Multi-Disease Medical Imaging Classification System combining three independently trained EfficientNet transfer-learning models (Brain MRI, Chest X-Ray, Chest CT) into a single Gradio web application.

Live deployment: huggingface.co/spaces/PhilopateerSharl/radiology_assistant

2. Project Overview

This project consists of **three independent deep learning notebooks**, each training a convolutional neural network (based on an EfficientNet backbone pretrained on ImageNet) to classify a different type of medical scan:

#	Notebook	Modality	Task	Backbone
1	EfficientNetB0_Brain_Tumor_MRI_Classification_3.ipynb	Brain MRI	4-class tumor classification	EfficientNetB0
2	covid_19.ipynb	Chest CT scan	Binary COVID-19 detection	EfficientNetB4*
3	Xray_Pneumonia_EfficientNetB3.ipynb	Chest X-Ray	Binary pneumonia detection	EfficientNetB3

* The notebook’s title cell and section headers describe the project as “COVID-19 Detection using **EfficientNetB3**,” but the code itself imports and instantiates `EfficientNetB4`, uses a 380×380 input size (EfficientNetB4’s native resolution), and the deployed application’s inline comments explicitly confirm `EfficientNetB4`. This documentation follows the architecture actually implemented in code.

The three trained models (`best_brain_tumor_model.keras`, `x-ray.keras`, `Covid_19.keras`) are combined into a single production web application, **MediScan AI**, built with **Gradio** and deployed on **Hugging Face Spaces**. The app lets a user select a scan type, upload/photograph a medical image, and receive an AI-generated diagnosis, a confidence score, a severity rating, and plain-language medical guidance. It also includes an integrated LLM-powered chatbot (via the **Groq API**, using `llama-3.3-70b-versatile`) that can answer questions about the three supported conditions, and it is aware of the user’s most recent scan result.

3. Problem Statement

Manual interpretation of radiological images (MRI, X-ray, CT) requires trained specialists and is time-consuming, which can delay diagnosis — particularly in resource-constrained or high-volume clinical settings. Three specific, high-impact diagnostic tasks are addressed:

- **Brain tumors** (glioma, meningioma, pituitary tumor) are difficult to detect early, and misclassification (e.g., a true tumor being called “no tumor”) carries serious clinical risk.
- **Pneumonia** requires timely diagnosis to begin correct antibiotic/antiviral treatment and reduce complications.
- **COVID-19**, particularly during periods of high caseload, benefits from CT-based screening as a complement to PCR/antigen testing.

The project explores whether transfer learning with EfficientNet architectures can produce automated, reasonably accurate screening tools for all three tasks, and whether such models can be unified into one accessible, patient-facing interface.

4. Objectives

Per-notebook objectives (as stated in the notebooks): - Load and preprocess medical images (MRI / CT / X-ray) for each condition. - Apply appropriate data augmentation to improve generalization. - Build EfficientNet-based transfer-learning classifiers. - Train models in two phases: head-only training followed by fine-tuning. - Evaluate performance using accuracy, precision, recall, F1-score, confusion matrices, and ROC-AUC. - Visualize training curves (accuracy/loss) and classification results. - Save the best-performing model checkpoint for deployment.

Project-level (deployment) objectives: - Unify all three trained models into a single diagnostic web application. - Provide patients/users with an understandable diagnosis, confidence level, and recommended next steps. - Offer an AI chatbot for follow-up questions about the diagnosed condition. - Maintain a session-based scan history log. - Clearly disclose that the tool is for educational/research purposes and is not a medical device.

5. Dataset Description

Dataset	Source	Classes	Total Images	Train	Val	Test
Brain Tumor MRI Dataset	Kaggle (brain-tumor-mri-dataset)	glioma , meningioma , notumor , pituitary	7,200	5,600 (1,400/class)	— (test set used for validation during training)	1,600 (400/class)
SARS-CoV-2 CT-Scan Dataset	Kaggle, plameneduardo/sarscov2-ctscan-dataset	COVID , non-COVID	2,481	1,744	496	241
Chest X-Ray Images (Pneumonia)	Kaggle, paultimothymooney/chest-xray-pneumonia (author: Paul Mooney)	NORMAL , PNEUMONIA	5,840 (train dir) + 624 (test dir)	4,173	1,043	624

Notes: - The Brain Tumor dataset is **perfectly class-balanced** (1,400 train / 400 test images per class), so no class weighting was required. - The COVID-19 dataset was split 70% / 20% / 10% (train/val/test) at the batch level (batch_size=16), which produced the exact image counts above (1,744 / 496 / 241). - The Pneumonia dataset uses Kaggle’s native train / val / test folder structure; because the native val folder is extremely small, an 80/20 split of the train folder was used instead (validation_split=0.2), and the original test folder (624 images) was kept fully independent for final evaluation. - Both binary datasets (COVID, Pneumonia) show mild class imbalance and were corrected using sklearn.utils.class_weight.compute_class_weight(class_weight="balanced", ...).

6. Data Preprocessing

Step	Brain Tumor (EfficientNetB0)	COVID CT (EfficientNetB4)	Pneumonia X-Ray (EfficientNetB3)
Image loading	ImageDataGenerator.flow_from_directory	tf.keras.utils.image_dataset_from_directory	tf.keras.utils.image_dataset_from_directory
Input resolution	224 × 224 × 3	380 × 380 × 3	384 × 384 × 3
Batch size	32	16	16
Pixel scaling	efficientnet.preprocess_input applied via preprocessing_function in the generator	efficientnet.preprocess_input applied as a layer inside the model graph	efficientnet.preprocess_input applied as a layer inside the model graph, after the augmentation layer
Label encoding	Categorical (one-hot, 4 classes)	Binary (label_mode="binary")	Binary (label_mode="binary")
Class imbalance handling	Not needed (balanced dataset)	class_weight="balanced" passed to model.fit	class_weight="balanced" passed to model.fit
Pipeline optimization	N/A (generator-based)	.cache().prefetch(AUTOTUNE)	.prefetch(AUTOTUNE)
Reproducibility	tf.random.set_seed(42) , np.random.seed(42) , seed=42 in generators	seed not explicitly fixed at the dataset-split step	seed=42 in image_dataset_from_directory splits

7. Exploratory Data Analysis

EDA in these notebooks is primarily visual rather than statistical:

- **Brain Tumor notebook:** prints the unzipped folder tree (depth-limited) to confirm dataset structure and per-class image counts, and prints the exact Found 5,600 / 1,600 images belonging to 4 classes confirmation from the data generators.
- **COVID-19 notebook:** prints Found 2,481 files belonging to 2 classes: ['COVID', 'non-COVID'], and computes/prints the class-weight dictionary to check for imbalance.
- **Pneumonia notebook:** walks the downloaded Kaggle directory to print its folder structure, and prints the training/validation/test split counts returned by image_dataset_from_directory (Found 5,216 files ... Using 4,173 for training , etc.).

No tabular/statistical EDA (e.g., pixel-intensity distributions, image-size histograms) is present in any notebook; class balance was assessed purely from file counts.

8. Data Augmentation

Notebook	Augmentation applied	Method
Brain Tumor	Horizontal flip, rotation ($\pm 15^\circ$), width/height shift ($\pm 10\%$), shear (10%), zoom (10%), brightness ($\pm 20\%$)	ImageDataGenerator parameters — training data only, never applied to the test generator
COVID-19 CT	None. No augmentation layer or generator augmentation is used; class imbalance is instead handled via computed class weights	—
Pneumonia X-Ray	Random rotation (± 0.05), random zoom (10%), random contrast (10%)	tf.keras.Sequential([layers.RandomRotation, layers.RandomZoom, layers.RandomContrast]), applied as the first layer inside the model graph itself (so augmentation happens on-the-fly at training time and automatically switches off during inference)

A 4x4 grid of augmented sample images per class is generated and saved (results/sample_images.png) in the Brain Tumor notebook; the COVID and Pneumonia notebooks each display a 3x3 grid of raw (unaugmented) sample images with their class labels for a quick visual sanity check.

9. Model Architecture(s)

All three models follow the same general transfer-learning pattern: a frozen/partially-frozen **EfficientNet** backbone (ImageNet weights) → **GlobalAveragePooling2D** → **BatchNormalization** → **Dense(256, ReLU)** → **Dropout** → **classification head**.

9.1 Brain Tumor Model — EfficientNetB0

```
Input (224, 224, 3)
→ EfficientNetB0 (ImageNet weights, 238 layers, no top)
→ GlobalAveragePooling2D → (1280,)
→ BatchNormalization
→ Dense(256, activation="relu")
→ Dropout(0.5)
→ Dense(4, activation="softmax") → [glioma, meningioma, notumor, pituitary]
```

- Total parameters: **4,383,675**
- Trainable parameters (Phase 1, backbone frozen): **331,524**
- Design rationale (from the notebook): GAP reduces overfitting vs. Flatten ; BatchNorm stabilizes/speeds up learning; Dropout(0.5) was chosen as optimal for medical imaging.

9.2 COVID-19 CT Model — EfficientNetB4

```
Input (380, 380, 3)
→ preprocess_input (EfficientNet scaling, inside the graph)
→ EfficientNetB4 (ImageNet weights, no top)
→ GlobalAveragePooling2D
→ BatchNormalization
→ Dropout(0.4)
→ Dense(256, activation="relu")
→ Dropout(0.3)
→ Dense(1, activation="sigmoid") → P(non-COVID)
```

- Total parameters: **18,140,256**
- Trainable parameters (Phase 1, backbone frozen): **462,849**
- Non-trainable parameters: **17,677,407**

9.3 Pneumonia X-Ray Model — EfficientNetB3

```
Input (384, 384, 3)
→ Data Augmentation (RandomRotation, RandomZoom, RandomContrast) [inside the graph]
→ preprocess_input (EfficientNet scaling, inside the graph)
```

```
→ EfficientNetB3 (ImageNet weights, no top)
→ GlobalAveragePooling2D
→ BatchNormalization
→ Dropout(0.4)
→ Dense(256, activation="relu")
→ Dropout(0.3)
→ Dense(1, activation="sigmoid")    → P(PNEUMONIA)
```

- Total parameters: **11,183,408**
- Trainable parameters (Phase 1, backbone frozen): **396,801**
- Non-trainable parameters: **10,786,607**

10. Training Process

All three models use a **two-phase transfer-learning strategy**: (1) train only the custom classification head while the backbone is frozen, then (2) unfreeze part or all of the backbone and fine-tune end-to-end at a much lower learning rate to avoid catastrophic forgetting of ImageNet features.

Phase	Brain Tumor (EfficientNetB0)	COVID CT (EfficientNetB4)	Pneumonia X-Ray (EfficientNetB3)
Phase 1 (head only)	15 epochs, backbone fully frozen (238 layers), LR = 1e-3	7 epochs, backbone fully frozen, LR = 1e-4	25 epochs, backbone fully frozen, LR = 1e-4
Phase 2 (fine-tune)	35 epochs, first 100 layers frozen / top 138 layers unfrozen, LR = 1e-5	3 epochs, last 80 layers unfrozen, LR = 1e-5	10 epochs, last 50 layers unfrozen, LR = 1e-5
Extra refinement	Additional “Enhanced Phase 2” : 20 more epochs, only first 50 layers frozen (188 layers unfrozen), label smoothing reduced 0.10 → 0.05, to specifically fix pituitary→“no tumor” false negatives	—	—

Class weighting (`class_weight="balanced"`) was applied during `model.fit` for the COVID and Pneumonia models to compensate for mild class imbalance; the Brain Tumor dataset needed no weighting since it is already balanced.

All models were trained on a GPU-accelerated runtime (TensorFlow 2.20.0, CUDA-enabled GPU confirmed in all three notebooks’ environment checks).

11. Hyperparameters

Hyperparameter	Brain Tumor	COVID-19 CT	Pneumonia X-Ray
Image size	224×224	380×380	384×384
Batch size	32	16	16
Phase-1 epochs	15	7	25
Phase-2 epochs	35 (+20 enhanced)	3	10
Phase-1 LR	1e-3	1e-4	1e-4
Phase-2 LR	1e-5	1e-5	1e-5
Dropout	0.5 (single)	0.4, 0.3	0.4, 0.3
Label smoothing	0.10 → 0.05 (enhanced)	—	—
Random seed	42	42 (partial)	42

Shared training callbacks (all three notebooks): `ModelCheckpoint` (saves the best model on `val_accuracy` for Brain Tumor, or `val_auc` for COVID/Pneumonia), `ReduceLR0nPlateau` (factor 0.5 for Brain Tumor / 0.2 for COVID & Pneumonia, patience 2-4 epochs on `val_loss`), and `EarlyStopping` (patience 8-12 epochs, `restore_best_weights=True`).

12. Loss Function and Optimizer

Model	Optimizer	Loss Function
Brain Tumor	Adam	<code>CategoricalCrossentropy(label_smoothing=0.1)</code> , reduced to <code>0.05</code> in the enhanced fine-tuning pass
COVID-19 CT	Adam	<code>binary_crossentropy</code>
Pneumonia X-Ray	Adam	<code>binary_crossentropy</code>

All three models track `accuracy` as a training metric; COVID and Pneumonia additionally track `tf.keras.metrics.AUC(name="auc")` during training.

13. Evaluation Metrics

- **Accuracy** — overall proportion of correct predictions.

- **Precision, Recall, F1-score** — per class, via `sklearn.metrics.classification_report`.
- **Confusion Matrix** — raw counts (and row-normalized percentages for the Brain Tumor model).
- **ROC-AUC** — one-vs-rest per class for the 4-class Brain Tumor model; single binary ROC-AUC for COVID and Pneumonia.
- **Custom composite “Ranking Score”** (COVID notebook only): $0.4 \times \text{Accuracy} + 0.3 \times \text{F1} + 0.3 \times \text{ROC-AUC}$, intended as a single comparable score across model iterations.

Recall was explicitly called out in the Brain Tumor notebook as the *most clinically important metric*, since a false negative (missing a real tumor) is more dangerous than a false positive.

14. Results

14.1 Brain Tumor MRI Classification (EfficientNetB0)

Test set: 1,600 images (400 per class).

Class	Precision	Recall	F1-score	Support
Glioma	0.9940	0.8350	0.9076	400
Meningioma	0.9116	0.9800	0.9446	400
No Tumor	0.9324	1.0000	0.9650	400
Pituitary	0.9852	0.9975	0.9913	400
Overall Accuracy			95.31%	1,600
Macro avg	0.9558	0.9531	0.9521	1,600

- Per-class AUC: Glioma 0.9695, Meningioma 0.9948, No Tumor 0.9990, Pituitary 0.9999 → **Macro AUC = 0.9908**
- After the additional “Enhanced Fine-Tuning” pass (targeting pituitary → no-tumor false negatives): **Accuracy improved to 95.63%**, Loss dropped to 0.3628 (+0.31 percentage points over the earlier best checkpoint).
- The weakest metric overall is **glioma recall (83.5%)**, indicating glioma is the class most often confused with another tumor type — a natural target for future improvement.

14.2 COVID-19 CT Classification (EfficientNetB4)

Test set: 241 CT images (126 COVID, 115 non-COVID).

Class	Precision	Recall	F1-score	Support
COVID	0.96	0.87	0.91	126
non-COVID	0.87	0.97	0.91	115
Overall Accuracy			91.29%	241
Macro avg	0.92	0.92	0.91	241

- **ROC-AUC: 97.14%**
- **Composite Ranking Score: 0.9306** ($0.4 \cdot \text{Acc} + 0.3 \cdot \text{F1} + 0.3 \cdot \text{AUC}$)
- The model is highly precise for COVID (96%) but has a comparatively lower COVID recall (87%), meaning some true COVID cases are missed — an important limitation to flag for a screening tool.

14.3 Pneumonia X-Ray Classification (EfficientNetB3)

Test set: 624 X-ray images (234 NORMAL, 390 PNEUMONIA).

Class	Precision	Recall	F1-score	Support
NORMAL	0.89	0.82	0.85	234
PNEUMONIA	0.90	0.94	0.92	390
Overall Accuracy			89.42%	624
Macro avg	0.89	0.88	0.89	624

- **ROC-AUC: 95.15%**
- The Pneumonia notebook’s own written “Results” narrative reports slightly higher headline numbers (Accuracy 91.51%, Precision 91.00%, Recall 95.90%, F1 93.38%, ROC-AUC 96.32%). This documentation reports the figures reproduced from the notebook’s actual `classification_report` execution log for traceability; the small gap between the two (~2 points) most likely reflects the difference between the best checkpoint saved by `ModelCheckpoint` during training and the final in-session evaluation run. Both figures indicate strong, consistent performance in the low-to-mid 90s%.
- A small illustrative inference demo (1 NORMAL + 1 PNEUMONIA sample image) correctly classified both cases (2/2, 100%) — a qualitative sanity check, not a statistically meaningful metric.

14.4 Cross-Model Summary

Model	Task	Accuracy	ROC-AUC (macro/binary)	F1 (macro)

Brain Tumor (EfficientNetB0)	4-class MRI classification	95.31% (95.63% enhanced)	0.9908	0.9521
COVID-19 (EfficientNetB4)	Binary CT classification	91.29%	0.9714	0.91
Pneumonia (EfficientNetB3)	Binary X-ray classification	89.42%	0.9515	0.89

15. Visualizations

Notebook	Plots generated
Brain Tumor	4×4 grid of augmented sample images by class (<code>sample_images.png</code>); combined accuracy/loss curves across both training phases with a marker at the fine-tuning start (<code>training_history.png</code>); dual confusion matrix (raw counts + row-normalized %) (<code>confusion_matrix.png</code>); grouped bar chart of per-class precision/recall/F1 with a 95% reference line (<code>class_metrics.png</code>); one-vs-rest ROC curves for all 4 classes with AUC in the legend (<code>roc_curves.png</code>); a small gallery of specific misclassified (false-negative) images for manual error analysis
COVID-19	3×3 grid of raw sample CT images; training/validation accuracy curve; training/validation loss curve; confusion matrix heatmap (Seaborn)
Pneumonia	3×3 grid of raw sample X-ray images; training/validation accuracy curve; training/validation loss curve; confusion matrix heatmap (Seaborn); side-by-side inference panel showing 2 sample test images with true label, predicted label, confidence, and correctness indicator

Only the Brain Tumor notebook persists its figures to disk (`./results/`); the COVID and Pneumonia notebooks display plots inline via `plt.show()` without saving them.

16. Inference / Prediction Process

Within the notebooks: - The Brain Tumor notebook defines a full single-image inference pipeline: load & resize to 224×224 → `EfficientNet preprocess_input` → predict → return predicted class, confidence, and a bar chart of all class probabilities. It also includes an experimental **safety layer** (`is_brain_scan`) that uses a separate pretrained `MobileNetV2` plus a grayscale heuristic to flag images that are unlikely to be brain MRIs before running the tumor classifier, and a `get_patient_insight()` helper that turns the raw prediction into a plain-language explanation with a medical disclaimer.

In the deployed application (`app.py` , **Hugging Face Space): 1. All three `.keras` models are loaded once at app startup. 2. The user selects a scan type (Brain MRI / Chest X-Ray / CT Scan), enters basic patient info (name, age, sex), and uploads or photographs an image. 3. The image is resized to the correct input size for the selected model (224×224 for MRI, 384×384 for X-ray, 380×380 for CT) and converted to a NumPy array. 4. The appropriate model runs a forward pass: - MRI → 4-way softmax → `argmax` → tumor class. - X-ray / CT → single sigmoid neuron → thresholded at 0.5. 5. A **probability safety-net function** (`_as_probabilities` / `_as_probability`) checks whether the raw model output already looks like a valid probability distribution; if not, it applies softmax/sigmoid manually. This was added specifically to fix a bug where some exported models returned raw logits, producing invalid confidence values (e.g., “1090%”). 6. The result is rendered as a color-coded card (severity level, description, recommended action, low-confidence warning if <70%) plus a confidence-breakdown bar chart and an SVG donut chart. 7. The diagnosis is stored in a session-level “current scan” object and appended to a session scan-history table. 8. An integrated chatbot (Groq `llama-3.3-70b-versatile`) can answer follow-up questions about the three supported conditions and is automatically primed with the user’s latest scan result for context-aware answers.**

17. Libraries and Dependencies

Training (across the three notebooks): - `tensorflow` / `keras` (2.20.0) — model building, training - `scikit-learn` — metrics (`classification_report` , `confusion_matrix` , `roc_curve` , `auc` , `roc_auc_score` , `precision_score` , `recall_score` , `f1_score` , `accuracy_score` , `compute_class_weight` , `label_binarize`) - `numpy` , `pandas` — data handling - `matplotlib` , `seaborn` — visualization - `Pillow` (PIL) — image loading - `tqdm` — progress bars - `kagglehub` — dataset download (COVID and Pneumonia notebooks) - `google.colab` (`drive.mount`) — model persistence to Google Drive (Pneumonia notebook)

Deployment (Hugging Face Space, `requirements.txt`): - `gradio` — web application framework/UI (SDK version 6.19.0) - `groq` — LLM chatbot API client - `numpy` - `tensorflow-cpu` — CPU-only inference runtime - `pillow`

18. Project Folder Structure

The notebooks and the Hugging Face Space were not provided as a single connected repository; the structure below is **reconstructed** from the file-saving code in the notebooks and the file list of the deployed Space.

```
radiology-assistant/
├── notebooks/
│   ├── EfficientNetB0__Brain_Tumor_MRI_Classification.ipynb
│   ├── covid_19.ipynb
│   └── Xray_Pneumonia_EfficientNetB3.ipynb
├── results/
│   ├── sample_images.png
│   ├── training_history.png
│   └── confusion_matrix.png
└── # Brain Tumor notebook output only
```

```

├── class_metrics.png
├── roc_curves.png
├── models/
│   ├── best_brain_tumor_model.keras          # Locally-saved checkpoints
│   ├── enhanced_sensitivity_model.keras      # Brain Tumor, post enhanced fine-tuning
│   ├── best_model.keras                     # COVID-19
│   └── best_efficientnetb3.keras             # Pneumonia
├── deployment/
│   ├── app.py                               # Hugging Face Space: "radiology_assistant"
│   ├── requirements.txt                     # Gradio application (MediScan AI)
│   ├── README.md                           # Space metadata (sdk: gradio)
│   ├── best_brain_tumor_model.keras
│   ├── x-ray.keras                          # Pneumonia model, renamed for deployment
│   └── Covid_19.keras                       # COVID-19 model, renamed for deployment

```

Datasets themselves are **not stored in the repository** — they are downloaded at runtime from Kaggle (via direct file upload/unzip for the Brain Tumor dataset, and via `kagglehub.dataset_download()` for the COVID and Pneumonia datasets).

19. Challenges Encountered

- **Class imbalance** in the COVID and Pneumonia datasets was addressed with `compute_class_weight(class_weight="balanced", ...)`; the Brain Tumor dataset avoided this issue entirely by being pre-balanced.
- **Catastrophic forgetting risk** during fine-tuning was mitigated in all three notebooks by using a very low learning rate (1e-5) and by unfreezing only part of the backbone rather than the whole network at once.
- **Specific false-negative pattern** in the Brain Tumor model (pituitary tumors occasionally misclassified as “no tumor”) was diagnosed via manual error analysis (indexing and visualizing the exact misclassified images) and addressed with an additional targeted fine-tuning pass (deeper unfreezing + lower label smoothing).
- **Inconsistent/anomalous `val_auc` logging** was observed during Pneumonia model training (`val_auc: 0.0000e+00` in several epochs), while the same AUC metric computed correctly (0.9556) during the final `model.evaluate()` call on the test set — suggesting a metric-state reset artifact during training rather than a genuine model issue.
- **Class-index ambiguity for binary datasets:** because `image_dataset_from_directory` assigns labels alphabetically, the meaning of “class 0” vs. “class 1” is not obvious from the code alone (e.g., for the CT scans, `COVID` sorts before `non-COVID`). The deployed app explicitly documents this with a `CT_FLIP` safety flag and instructions to verify against a known-COVID image before trusting the output.
- **Raw logits vs. probabilities bug in production:** some exported `.keras` models returned unnormalized outputs when reloaded, producing invalid confidence percentages (observed values over 1000%) in the deployed app. This was fixed with defensive `_as_probability(s)` helper functions that detect and correct for this at inference time.
- **Environment portability:** the Brain Tumor notebook needed separate path-handling logic for Google Colab (`/content/...` after manual `.zip` upload), Kaggle Notebooks, and local execution, since the dataset was not always downloaded the same way.
- **Discrepancy between narrative and logged results** (Pneumonia notebook): the notebook’s own written results summary (accuracy 91.51%) is a few points higher than the metrics reproducible from the notebook’s own final `classification_report` output (89.42%), most likely because the narrative reflects the best checkpoint saved during training rather than the specific run shown in the visible execution log.

20. Future Improvements

- Add **explainability** (e.g., Grad-CAM heatmaps) so users and clinicians can see which regions of the scan influenced the prediction, rather than relying on a confidence score alone.
- Replace the heuristic `MobileNetV2` + grayscale “is this a brain scan?” check in the Brain Tumor notebook with a dedicated, properly trained “relevant scan vs. irrelevant image” classifier.
- Resolve the `CT_FLIP` class-order ambiguity permanently by explicitly encoding/verifying class-to-index mapping at training time rather than relying on alphabetical folder ordering plus a manual flag.
- Expand each binary classifier (COVID, Pneumonia) toward **multi-class differential diagnosis** (e.g., bacterial vs. viral pneumonia, or other lung pathologies) instead of a single disease vs. normal.
- Perform **k-fold cross-validation** and evaluate on external, out-of-distribution datasets to better estimate real-world generalization.
- Investigate the **glioma recall gap** (83.5%) in the Brain Tumor model with further error analysis and targeted data augmentation or oversampling.
- Add persistent (database-backed) storage for patient scan history in the deployed app — the current implementation resets whenever the Hugging Face Space restarts or goes to sleep.
- Support DICOM file input directly, in addition to standard JPG/PNG uploads, to better match real clinical workflows.
- Quantize/optimize the models (e.g., TensorFlow Lite) for faster CPU inference or edge/mobile deployment.
- Add automated regression tests around the app’s probability-normalization logic to catch logits/probability bugs like the one already fixed, before they reach production.

21. Conclusion

This project demonstrates a complete, end-to-end applied deep learning workflow across three distinct medical imaging tasks — brain tumor MRI classification, COVID-19 CT screening, and pneumonia X-ray detection — using a consistent transfer-learning methodology built on EfficientNet backbones (B0, B4, and B3 respectively). All three models achieved strong test-set performance (89–96% accuracy, 0.95–0.99 ROC-AUC), with particular attention paid to recall on the medically riskier error type (missed true positives) in the Brain Tumor notebook.

Beyond the individual models, the project’s most notable achievement is unifying all three into a single, polished, patient-facing web application

(**MediScan AI**) deployed on Hugging Face Spaces — complete with a confidence-aware UI, a context-aware medical chatbot, and clear educational disclaimers. The documented challenges (class-order ambiguity, logits-vs-probability bugs, false-negative patterns) and their fixes reflect a realistic, iterative engineering process rather than a single clean training run, and the future-improvements list outlines a credible path toward a more clinically robust system.

Appendix: Data Sources Referenced in the Notebooks

- Brain Tumor MRI Dataset — Kaggle (`brain-tumor-mri-dataset`)
- SARS-CoV-2 CT-Scan Dataset — Kaggle, [plameneduardo/sarscov2-ctscan-dataset](#)
- Chest X-Ray Images (Pneumonia) — Kaggle, `paultimothymooney/chest-xray-pneumonia` , author Paul Mooney

Not specified in the notebooks: exact license terms of each dataset, hardware specifications beyond “GPU detected,” and the total wall-clock training time for each model.